

Операционные системы

Анализ файловой структуры UNIX. Команды для работы с файлами и каталогами

Савостин Владислав Михайлович

2026-03-16

Цели и задачи работы

Процесс выполнения лабораторной работы

Выводы по проделанной работе

1. Цели и задачи работы

Ознакомление с файловой системой Linux, её структурой, именами и содержанием каталогов. Приобретение практических навыков по применению команд для работы с файлами и каталогами, по управлению процессами, по проверке использования диска и обслуживанию файловой системы.

- 1 Выполнить приимеры
- 2 Выполнить дествия по работе с каталогами и файлами
- 3 Выполнить действия с правами доступа
- 4 Получить дополнительные сведения при помощи справки по командам.

2. Процесс выполнения лабораторной работы

```
vmsavostin@vmsavostin:~$ touch abc1
vmsavostin@vmsavostin:~$ cp abc1 april
vmsavostin@vmsavostin:~$ cp abc1 may
vmsavostin@vmsavostin:~$
^[[200~mkdir monthly
^[[201~vmsavostin@vmsavostin:~$ mkdir monthlymonthly
vmsavostin@vmsavostin:~$ cp april may monthly
vmsavostin@vmsavostin:~$ cp monthly/may monthly/june
vmsavostin@vmsavostin:~$ ls monthly
april  june  may
vmsavostin@vmsavostin:~$ ls monthly
april  june  may
vmsavostin@vmsavostin:~$ cp -r monthly monthly.00
vmsavostin@vmsavostin:~$ cp -r monthly.00 /tmp
vmsavostin@vmsavostin:~$
```

Рисунок 1: Выполнение примеров

```
vmsavostin@vmsavostin:~$  
vmsavostin@vmsavostin:~$ mv april july  
vmsavostin@vmsavostin:~$ mv july monthly.00  
vmsavostin@vmsavostin:~$ ls monthly.00  
april july june may  
vmsavostin@vmsavostin:~$ mv monthly.00 monthly.01  
vmsavostin@vmsavostin:~$ mkdir reports  
vmsavostin@vmsavostin:~$ mv monthly.01 reports  
vmsavostin@vmsavostin:~$ mv reports/monthly.01 reports/monthly  
vmsavostin@vmsavostin:~$
```

Рисунок 2: Выполнение примеров

```
vm savostin@vm savostin:~$  
vm savostin@vm savostin:~$ touch may  
vm savostin@vm savostin:~$ ls -l may  
-rw-r--r--. 1 vm savostin vm savostin 0 Mar 16 14:58 may  
vm savostin@vm savostin:~$ chmod u+x may  
vm savostin@vm savostin:~$ ls -l may  
-rwxr--r--. 1 vm savostin vm savostin 0 Mar 16 14:58 may  
vm savostin@vm savostin:~$ chmod u-x may  
vm savostin@vm savostin:~$ ls -l may  
-rw-r--r--. 1 vm savostin vm savostin 0 Mar 16 14:58 may  
vm savostin@vm savostin:~$ chmod g-r,o-r monthly  
vm savostin@vm savostin:~$ chmod g+w abc1  
vm savostin@vm savostin:~$
```

Рисунок 3: Выполнение примеров

Создание директорий и копирование файлов

```
vmsavostin@vmsavostin:~$  
vmsavostin@vmsavostin:~$ cp /usr/include/linux/sysinfo.h ~  
vmsavostin@vmsavostin:~$ mv sysinfo.h equipment  
vmsavostin@vmsavostin:~$ mkdir ski.plases  
vmsavostin@vmsavostin:~$ mv equipment ski.plases/  
vmsavostin@vmsavostin:~$ mv ski.plases/equipment ski.plases/equi  
plist  
vmsavostin@vmsavostin:~$ touch abc1  
vmsavostin@vmsavostin:~$ cp abc1 ski.plases/equiplist2  
vmsavostin@vmsavostin:~$ cd ski.plases/  
vmsavostin@vmsavostin:~/ski.plases$ mkdir equipment  
vmsavostin@vmsavostin:~/ski.plases$  
vmsavostin@vmsavostin:~/ski.plases$ mv equiplist equipment/  
vmsavostin@vmsavostin:~/ski.plases$ mv equiplist2 equipment/  
vmsavostin@vmsavostin:~/ski.plases$ cd  
vmsavostin@vmsavostin:~$ mkdir newdir  
vmsavostin@vmsavostin:~$ mv newdir ski.plases/  
vmsavostin@vmsavostin:~$ mv ski.plases/newdir/ ski.plases/plans  
vmsavostin@vmsavostin:~$
```

Рисунок 4: Работа с каталогами

Работа с командой chmod

```
vmsavostin@vmsavostin:~$ mkdir australia play
vmsavostin@vmsavostin:~$ touch my_os feathers
vmsavostin@vmsavostin:~$ chmod 744 australia/
vmsavostin@vmsavostin:~$ chmod 711 play/
vmsavostin@vmsavostin:~$ chmod 544 my_os
vmsavostin@vmsavostin:~$ chmod 664 feathers
vmsavostin@vmsavostin:~$ ls -l
total 0
-rw-rw-r--. 1 vmsavostin vmsavostin 0 Mar 16 14:58 abc1
drwxr--r--. 1 vmsavostin vmsavostin 0 Mar 16 14:59 australia
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Desktop
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Documents
drwxr-xr-x. 1 vmsavostin vmsavostin 44 Mar 6 19:35 Downloads
-rw-rw-r--. 1 vmsavostin vmsavostin 0 Mar 16 14:59 feathers
drwxr-xr-x. 1 vmsavostin vmsavostin 74 Mar 6 19:25 git-extended
-rw-r--r--. 1 vmsavostin vmsavostin 0 Mar 16 14:58 may
drwx--x--x. 1 vmsavostin vmsavostin 24 Mar 16 14:57 monthly
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Music
-r-xr--r--. 1 vmsavostin vmsavostin 0 Mar 16 14:59 my_os
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Pictures
drwx--x--x. 1 vmsavostin vmsavostin 0 Mar 16 14:59 play
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Public
drwxr-xr-x. 1 vmsavostin vmsavostin 14 Mar 16 14:58 reports
drwxr-xr-x. 1 vmsavostin vmsavostin 48 Mar 6 19:32 site
drwxr-xr-x. 1 vmsavostin vmsavostin 28 Mar 16 14:59 ski.plases
drwx-----. 1 vmsavostin vmsavostin 8 Mar 6 19:32 snap
drwxr-xr-x. 1 vmsavostin vmsavostin 0 Mar 6 19:09 Templates
```

```

root:x:0:0:Super User:/root:/bin/bash
bin:x:1:1:bin:/bin:/usr/sbin/nologin
daemon:x:2:2:daemon:/sbin:/usr/sbin/nologin
adm:x:3:4:adm:/var/adm:/usr/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/usr/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/usr/sbin/nologin
operator:x:11:0:operator:/root:/usr/sbin/nologin
games:x:12:100:games:/usr/games:/usr/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/usr/sbin/nologin
nobody:x:65534:65534:Kernel Overflow User:/usr/sbin/nologin
dbus:x:81:81:System Message Bus:/usr/sbin/nologin
tss:x:59:59:Account used for TPM access:/usr/sbin/nologin
systemd-oom:x:999:999:systemd Userspace OOM Killer:/usr/sbin/nologin
polkitd:x:114:114>User for polkitd:/usr/sbin/nologin
systemd-coredump:x:998:998:systemd Core Dumper:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/usr/sbin/nologin
chrony:x:996:996:chrony system user:/var/lib/chrony:/usr/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/usr/sbin/nologin
systemd-resolve:x:193:193:systemd Resolver:/usr/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/usr/sbin/nologin
unbound:x:993:993:Unbound DNS resolver:/var/lib/unbound:/usr/sbin/nologin
clevis:x:992:992:Clevis Decryption Framework unprivileged user:/usr/sbin/nologin

```

```
vmsavostin@vmsavostin:~$  
vmsavostin@vmsavostin:~$ cp feathers file.old  
vmsavostin@vmsavostin:~$ mv file.old play/  
vmsavostin@vmsavostin:~$ mkdir fun  
vmsavostin@vmsavostin:~$ cp -R play/ fun/  
vmsavostin@vmsavostin:~$ mv fun/ play/games  
vmsavostin@vmsavostin:~$ chmod u-r feathers  
vmsavostin@vmsavostin:~$ cat feathers  
cat: feathers: Permission denied  
vmsavostin@vmsavostin:~$ cp feathers feathers2  
cp: cannot open 'feathers' for reading: Permission denied  
vmsavostin@vmsavostin:~$ chmod u+r feathers  
vmsavostin@vmsavostin:~$ chmod u-x play/  
vmsavostin@vmsavostin:~$ cd play/  
bash: cd: play/: Permission denied  
vmsavostin@vmsavostin:~$ chmod +x play/  
vmsavostin@vmsavostin:~$
```

Рисунок 7: Работа с файлами и правами доступа

```
MOUNT(8)                                     System Administration                                     MOUNT(8)

NAME
    mount - mount a filesystem

SYNOPSIS
    mount [-h|-V]

    mount [-l] [-t fstype]

    mount -a [-fFnrsvw] [-t fstype] [-O optlist]

    mount [-fnrsvw] [-o options] device|mountpoint

    mount [-fnrsvw] [-t fstype] [-o options] device mountpoint

    mount --bind|--rbind|--move olddir newdir

    mount --make-[shared|slave|private|unbindable|rshared|rslave|rprivate|runbindable] mountpoint

DESCRIPTION
    All files accessible in a Unix system are arranged in one big tree, the file hierarchy, rooted at /.
    These files can be spread out over several devices. The mount command serves to attach the filesystem
    found on some device to the big file tree. Conversely, the umount(8) command will detach it again. The
    filesystem is used to control how data is stored on the device or provided in a virtual way by network
    or other services.

    The standard form of the mount command is:

        mount -t type device dir

    This tells the kernel to attach the filesystem found on device (which is of type type) at the
    directory dir. The option -t type is optional. The mount command is usually able to detect a
    filesystem. The root permissions are necessary to mount a filesystem by default. See section

Manual page mount(8) line 1 (press h for help or q to quit)
```

```
FSCK(8)                                     System Administration                                     FSCK(8)

NAME
    fsck - check and repair a Linux filesystem

SYNOPSIS
    fsck [-lsAVRTMNP] [-r [fd]] [-C [fd]] [-t fstype] [filesystem...] [--] [fs-specific-options]

DESCRIPTION
    fsck is used to check and optionally repair one or more Linux filesystems. filesystem can be a device
    name (e.g., /dev/hdc1, /dev/sdb2), a mount point (e.g., /, /usr, /home), or a filesystem label or UUID
    specifier (e.g., UUID=8868abf6-88c5-4a83-98b8-bfc24057f7bd or LABEL=root). Normally, the fsck program
    will try to handle filesystems on different physical disk drives in parallel to reduce the total
    amount of time needed to check all of them.

    If no filesystems are specified on the command line, and the -A option is not specified, fsck will
    default to checking filesystems in /etc/fstab serially. This is equivalent to the -As options.

    The exit status returned by fsck is the sum of the following conditions:

    0
        No errors

    1
        Filesystem errors corrected

    2
        System should be rebooted

    4
        Filesystem errors left uncorrected

    8
        Operational error

Manual page fsck(8) line 1 (press h for help or q to quit)
```

```

MKFS(8)                                     System Administration                               MKFS(8)

NAME
    mkfs - build a Linux filesystem

SYNOPSIS
    mkfs [options] [-t type] [fs-options] device [size]

DESCRIPTION
    This mkfs frontend is deprecated in favour of filesystem specific mkfs.<type> utils.

    mkfs is used to build a Linux filesystem on a device, usually a hard disk partition. The device
    argument is either the device name (e.g., /dev/hda1, /dev/sdb2), or a regular file that shall contain
    the filesystem. The size argument is the number of blocks to be used for the filesystem.

    The exit status returned by mkfs is 0 on success and 1 on failure.

    In actuality, mkfs is simply a front-end for the various filesystem builders (mkfs.fstype) available
    under Linux. The filesystem-specific builder is searched for via your PATH environment setting only.
    Please see the filesystem-specific builder manual pages for further details.

OPTIONS
    -t, --type type
        Specify the type of filesystem to be built. If not specified, the default filesystem type
        (currently ext2) is used.

    fs-options
        Filesystem-specific options to be passed to the real filesystem builder.

    -V, --verbose
        Produce verbose output, including all filesystem-specific commands that are executed. Specifying
        this option more than once inhibits execution of any filesystem-specific commands. This is really
        only useful for testing.

Manual page mkfs(8) line 1 (press h for help or q to quit)
```

Рисунок 10: Команда mkfs

```
KILL(1)                                     User Commands                               KILL(1)

NAME
    kill - terminate a process

SYNOPSIS
    kill [-signal|-s signal|-p] [-q value] [-a] [--timeout milliseconds signal] [--] pid|name...

    kill -l [number|0xsigmask] | -L

    kill -d pid

DESCRIPTION
    The command kill sends the specified signal to the specified processes or process groups.

    If no signal is specified, the TERM signal is sent. The default action for this signal is to terminate the process. This signal should be used in preference to the KILL signal (number 9), since a process may install a handler for the TERM signal in order to perform clean-up steps before terminating in an orderly fashion. If a process does not terminate after a TERM signal has been sent, then the KILL signal may be used; be aware that the latter signal cannot be caught, and so does not give the target process the opportunity to perform any clean-up before terminating.

    Most modern shells have a builtin kill command, with a usage rather similar to that of the command described here. The --all, --pid, and --queue options, and the possibility to specify processes by command name, are local extensions.

    If signal is 0, then no actual signal is sent, but error checking is still performed.

ARGUMENTS
    The list of processes to be signaled can be a mixture of names and PIDs.

    pid
        Each pid can be expressed in one of the following ways:
```

Manual page kill(1) line 1 (press h for help or q to quit)

3. Выводы по проделанной работе

В ходе данной работы мы ознакомились с файловой системой Linux, её структурой, именами и содержанием каталогов. Научились совершать базовые операции с файлами, управлять правами их доступа для пользователя и групп. Ознакомились с Анализом файловой системы. А также получили базовые навыки по проверке использования диска и обслуживанию файловой системы.